

Spring Boot Hour 6 Relationships & JPA Traps

Spring Boot — 8-Hour Plan · HOUR 6

Relationships & the Famous JPA Traps

Goal: this is the hour your three saved LinkedIn scenario questions come from — `LazyInitializationException` in production, the million-row slowdown, and the silent overwrite. These are asked precisely because they filter people who've only done tutorials from people who understand the machine. After this hour, you're in the second group. Everything builds on Hour 5's entity states — keep the hospital in mind.

6.1 Relationships — connecting entities like tables connect

Real data is connected: a Department has many Employees; each Employee belongs to one Department. In the **database**, this is one foreign key column: `employee.department_id`. In **Java**, it's references between objects. JPA maps between the two:

```
@Entity
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne                                // MANY employees → ONE department
    @JoinColumn(name = "department_id")       // ← the actual FK column in the employee table
    private Department department;
}

@Entity
public class Department {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "department")       // ONE department → MANY employees
    private List<Employee> employees = new ArrayList<>();
}
```

Read the annotation from the perspective of the class it sits in: in `Employee`, "**many** of me relate **to one** Department" → `@ManyToOne`. In `Department`, "**one** of me relates **to many** Employees" → `@OneToMany`.

The part interviewers actually probe — who OWNS the relationship? The database has only ONE fact storing this link: the `department_id` column, and it lives in the **employee** table. So the `@ManyToOne` side (`Employee`) is the **owning side** — it holds the FK via `@JoinColumn`. The `@OneToMany` side is just a **mirror**: `mappedBy = "department"` says "I don't own anything; go look at the `department` field in `Employee` — that's where the real link lives." Forget `mappedBy` and Hibernate thinks these are two SEPARATE relationships and creates an extra join table — a classic bug.

Analogy: a child's school form has a column "father's name" (the FK — the child record OWNS the link). The father doesn't carry a list of children in his wallet; "his children" is just the mirror question — 'which forms mention me?' `mappedBy = "the truth is written on the child's form, not here."`

Other relationship types, quickly: `@OneToOne` (person ↔ passport, FK on one side), `@ManyToMany` (student ↔ course — no side can hold the FK, so a **join table** `student_course` holds pairs of IDs; JPA maps it with `@JoinTable`).

INTERVIEW CHECKPOINT

Q: "What does `mappedBy` mean?"  "It marks the inverse, non-owning side of a bidirectional relationship and points at the field on the owning side that holds the foreign key. Only the owning side's changes are written to the FK column; omitting `mappedBy` makes Hibernate treat it as two relationships and create a redundant join table."

Q: "Where does the foreign key live in `@OneToMany/@ManyToOne`?"  "Always on the many side — the `@ManyToOne` with `@JoinColumn` — because that's where the FK column physically is in the table."

6.2 LAZY vs EAGER — when do the related objects actually load?

Here's the question that creates all three of your LinkedIn traps. You run:

```
Department d = repo.findById(1L).get();
```

Should Hibernate ALSO load the department's 5,000 employees, right now? You might only need the department's name! Loading 5,000 objects for a name would be absurd. So JPA gives you a choice per relationship:

- **EAGER** — load the related objects **immediately**, in the same query (or an extra one right away).
- **LAZY** — load them **only if and when you actually touch them**.

```
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY) // load on touch
private List<Employee> employees;
```

How does LAZY even work? Hibernate is sneaky: instead of a real list, it puts a **proxy** in the field — a stand-in object that *looks* like a List but is actually empty, holding just a note: "if anyone calls me, run a *SELECT* and fetch the real employees." The moment you call `d.getEmployees().size()`, the proxy wakes up, **asks the session** to run the query, and fills itself.

Analogy: a restaurant menu with photos. EAGER = the waiter brings you every dish physically the moment you sit down — enormous waste if you only wanted tea. LAZY = you get the menu (proxy); a dish is cooked only when you point at it. Obviously smarter — but note the hidden dependency: pointing at the menu only works while the kitchen is still open. Hold that thought.

Defaults (memorize): @ManyToOne and @OneToOne → EAGER by default; @OneToMany and @ManyToMany → LAZY by default. (Memory hook: "...ToOne = one small object = eager by default; ...ToMany = potentially thousands = lazy by default.") Best practice you can state: **make everything LAZY explicitly** and fetch eagerly per-query when needed — you'll see how in 6.4.

💬 INTERVIEW CHECKPOINT

Q: "LAZY vs EAGER, and the defaults?" 🧠 "EAGER loads associations immediately with the parent; LAZY defers loading until first access, via a proxy that triggers a query through the open session. ToOne relationships default to EAGER, ToMany to LAZY. The recommendation is LAZY everywhere, fetching explicitly per use case."

6.3 🧨 LinkedIn Q13: "Lazy loading works in development but throws LazyInitializationException in production. Why?"

Now cash in everything from Hour 5. Watch the crime happen in slow motion:

```
@Service
public class DeptService {
    @Transactional
    public Department getDept(Long id) {
        return repo.findById(id).get(); // PERSISTENT; employees = sleeping proxy
    } // ← method ends → session CLOSES → DETACHED

@RestController
public class DeptController {
    @GetMapping("/dept/{id}")
    public List<Employee> emps(@PathVariable Long id) {
        Department d = service.getDept(id); // d is DETACHED here
        return d.getEmployees(); // ❗ proxy wakes up, asks the session...
    } // ...but the session is CLOSED.
    // LazyInitializationException!
}
```

The proxy's promise was "when touched, I'll ask **the session** to load the data." You touched it **after** the session died. The kitchen closed; you're pointing at the menu in the parking lot.

So why does it "work in development"? This is the sneaky part of the question. Spring Boot ships with a setting called **Open Session In View (OSIV)** — `spring.jpa.open-in-view=true` **by default** — which keeps the session open for the **entire HTTP request**, all the way through the controller and JSON serialization. With OSIV on, the parking lot doesn't exist: the kitchen stays open until the response is sent, so lazy touches in the controller quietly work. Many production setups **disable OSIV** (it holds DB connections too long, hurting throughput under load — and it hides exactly this bug). Different setting between environments → works on your laptop, explodes in prod. Mystery solved.

The three real fixes (know all, prefer the first two):

1. **Fetch what you need inside the transaction** — a JOIN FETCH query (next section) so employees are loaded *before* the session closes. No proxy left to wake.

2. **Return a DTO, not the entity** — build a plain response object (id, name, employee names) **inside** the @Transactional method, while everything is reachable. The controller gets simple data, no proxies at all. This is the production-grade pattern.
3. Widen the transaction to cover the access — works, but often just spreads the session around instead of fixing the design.

(The anti-fix to name-and-reject: switching to EAGER "to make the error go away" — it trades a visible exception for invisible performance damage everywhere.)

🗨 INTERVIEW CHECKPOINT — answer to Q13, ready to recite

☛ "LazyInitializationException means a lazy proxy was touched after its session closed — typically an entity accessed in the controller after the transactional method ended. It 'works in dev' usually because Open Session In View is enabled there, keeping the session open for the whole request, while production disables OSIV for connection-pool health. The proper fixes are fetching the needed associations inside the transaction with a fetch join or entity graph, or mapping to DTOs before the transaction ends — not switching to EAGER."

6.4 🍀 LinkedIn Q14: "A query performs well with 100 rows but becomes extremely slow with 1 million. Root cause?"

Two root causes to give — the first is THE expected answer.

Root cause #1 — the N+1 query problem. Innocent-looking code:

```
List<Department> depts = deptRepo.findAll();           // query 1: all departments
for (Department d : depts) {
    System.out.println(d.getEmployees().size());      // each touch wakes a LAZY proxy...
}
```

Count the SQL: **1** query for departments + **N** queries (one per department, fired by each proxy waking up) = **N+1 queries**. With 100 departments: 101 queries — imperceptible in dev. With 100,000: 100,001 network round-trips to the database. Each is fast; the *sum* is death. This is why "works with 100 rows, dies at scale" — **the cost was per-row all along**, dev data was just too small to feel it.

Analogy: sending a courier to fetch a list of 1,000 customer names, then sending him back out 1,000 separate times, once per customer, to ask each one's phone number. Each trip is 10 minutes. Nobody notices at 5 customers. At 1,000, your day is gone. The fix is obvious: ONE trip that collects names AND numbers together.

The fixes:

```
// Fix 1 - JOIN FETCH: one query, associations loaded together
@Query("SELECT DISTINCT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees();

// Fix 2 - @EntityGraph: same effect, no JPQL, works on derived methods
@EntityGraph(attributePaths = "employees")
List<Department> findAll();
```

Both produce **one** SQL join instead of N+1 round-trips. (Also note the diagnosis skill: `spring.jpa.show-sql=true` in dev and *count the queries* — seeing 101 SELECTs scroll past is how you catch this before prod does.)

Root cause #2 — unbounded fetching. Even with N+1 fixed, `findAll()` on a million-row table drags a million entities over the network into JVM memory. Nothing survives that. The fix is Hour 5's planted seed — **pagination**:

```
Page<Department> page = repo.findAll(PageRequest.of(0, 20)); // 20 rows, LIMIT in SQL
```

Plus, for truly large tables: proper **indexes** on filtered columns, and selecting only needed columns via DTO projections.

🗨 INTERVIEW CHECKPOINT — answer to Q14, ready to recite

☛ "Almost always the N+1 problem: a parent query plus one lazy-load query per row — invisible at 100 rows, catastrophic at a million because the cost is per-row. I'd confirm by logging SQL and counting queries, then fix with JOIN FETCH or @EntityGraph to load associations in one join. If the issue is sheer volume, the query must be paginated with Pageable and backed by proper indexes — `findAll()` on large tables should never exist."

6.5 🍀 LinkedIn Q15: "Two concurrent transactions update the same record; one silently overwrites the other. How to prevent it?"

Recognize this? It's your Java threading hour's **race condition** — the two ATMs and one bank account — reborn at the database level. The "lost update":

```
Row: product stock = 10
User A reads stock (10)      User B reads stock (10)
A computes 10 - 3 = 7        B computes 10 - 5 = 5
A writes 7                  B writes 5      ← A's update SILENTLY GONE
Reality should be 2. DB says 5. Nobody got an error.
```

The DB happily accepted both writes — each was individually valid. Two prevention strategies, and the names matter:

Strategy 1 — Optimistic locking (the default answer): `@Version`. Add one field:

```
@Entity
public class Product {
    @Id private Long id;
    private int stock;

    @Version                // ← that's the entire setup
    private Long version;
}
```

Now Hibernate changes every UPDATE to:

```
UPDATE product SET stock=7, version=6 WHERE id=1 AND version=5
--                                     ~~~~~ the magic
```

A reads (version 5). B reads (version 5). A commits → row becomes version 6. B commits → its WHERE clause looks for version **5**, finds **0 rows** → Hibernate throws `OptimisticLockException` → B's transaction rolls back, and your code retries or tells the user "data changed, please refresh." **The overwrite becomes impossible; the loser finds out instead of silently losing.**

Recognize the mechanism? It's **CAS from your threading hour** — "update only if it's still what I read, else fail and retry" — implemented with a WHERE clause. Same idea, different layer. Say that in an interview and watch the interviewer's eyebrows rise.

Strategy 2 — Pessimistic locking: lock the row while reading.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)    // → SELECT ... FOR UPDATE
Optional<Product> findWithLockById(Long id);
```

B's read now **blocks** until A's transaction finishes — conflicts prevented by exclusion rather than detection. Costs concurrency (rows locked = others wait; deadlock risk returns — bridge standoff, DB edition).

Which when? Same logic as CAS vs synchronized: *optimistic* when conflicts are rare (most web apps — pay only on actual conflict), *pessimistic* when conflicts are frequent or a retry is unacceptable (inventory counters, seat booking).

💬 INTERVIEW CHECKPOINT — answer to Q15, ready to recite

💡 "That's the lost-update problem. The standard prevention is optimistic locking: a `@Version` column that Hibernate includes in every UPDATE's WHERE clause — if another transaction bumped the version, zero rows match, an `OptimisticLockException` fires, and the losing transaction retries instead of silently overwriting. Where conflicts are frequent, pessimistic locking (`SELECT FOR UPDATE`) blocks the second reader instead, trading throughput for certainty."

6.6 The 30-second story for this hour

"Relationships map foreign keys: the `@ManyToOne` side owns the FK; `mappedBy` marks the mirror. Associations should be LAZY, loaded via proxies through the open session. From that, the three classic traps: touch a proxy after the session closes and you get `LazyInitializationException` — often masked in dev by `Open Session In View`; loop over lazy associations and you get N+1 queries — fixed by JOIN FETCH or entity graphs, plus pagination for volume; and concurrent updates cause lost updates — prevented by `@Version` optimistic locking, or pessimistic row locks when contention is high."

🔧 HOUR 6 DRILL — out loud

1. Where does the FK live, and which side owns the relationship? (many side, `@JoinColumn`; `@ManyToOne` owns)
2. What does `mappedBy` do and what if you forget it? (marks the mirror side; forget = redundant join table)
3. LAZY vs EAGER + defaults? (proxy-on-touch vs load-now; `ToOne` eager, `ToMany` lazy; prefer LAZY)

4. What IS a lazy proxy's promise? ("when touched, I'll ask the SESSION" — hence the trap)
 5. Explain LazyInitializationException end to end. (detached entity + touched proxy + closed session)
 6. Why does it work in dev but not prod? (OSIV default on vs disabled)
 7. Two proper fixes (and the anti-fix)? (JOIN FETCH/EntityGraph inside the transaction; DTOs; NOT blanket EAGER)
 8. What is N+1 and why invisible at 100 rows? (1 + one-query-per-row; per-row cost needs scale to hurt)
 9. Fixes for N+1 and for volume? (JOIN FETCH/@EntityGraph; Pageable + indexes)
 10. Lost update — what is it and the two defenses? (silent overwrite; @Version optimistic / pessimistic FOR UPDATE)
 11. How does @Version actually block the overwrite? (version in the WHERE clause → 0 rows → exception → retry)
 12. Bonus crossover: what threading concept is optimistic locking equivalent to? (CAS — compare-and-swap at the DB layer)
-

Next → Hour 7: Validation, Exception Handling & Logging — @Valid and the Bean Validation annotations, building the global @RestControllerAdvice handler, and SLF4J/Logback with log levels.